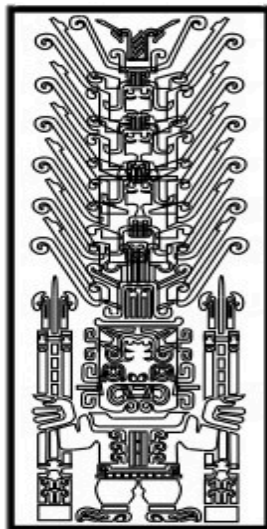


UNIVERSIDAD NACIONAL FEDERICO VILLARREAL
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ESCUELA PROFESIONAL: INGENIERÍA DE SISTEMAS



Sistema portátil de monitoreo de ritmo cardíaco con alerta automática mediante Bluetooth y aplicación móvil para contactos de emergencia

PRESENTADO POR:

Galvan Orellana Gabriel Felipe
Mantilla Isminio César Angelo
Ormeño Torres Fabio Jose
Saavedra Crispin Leonardo Jesus

Equipo: 2

DOCENTE: Hernan Villafuerte

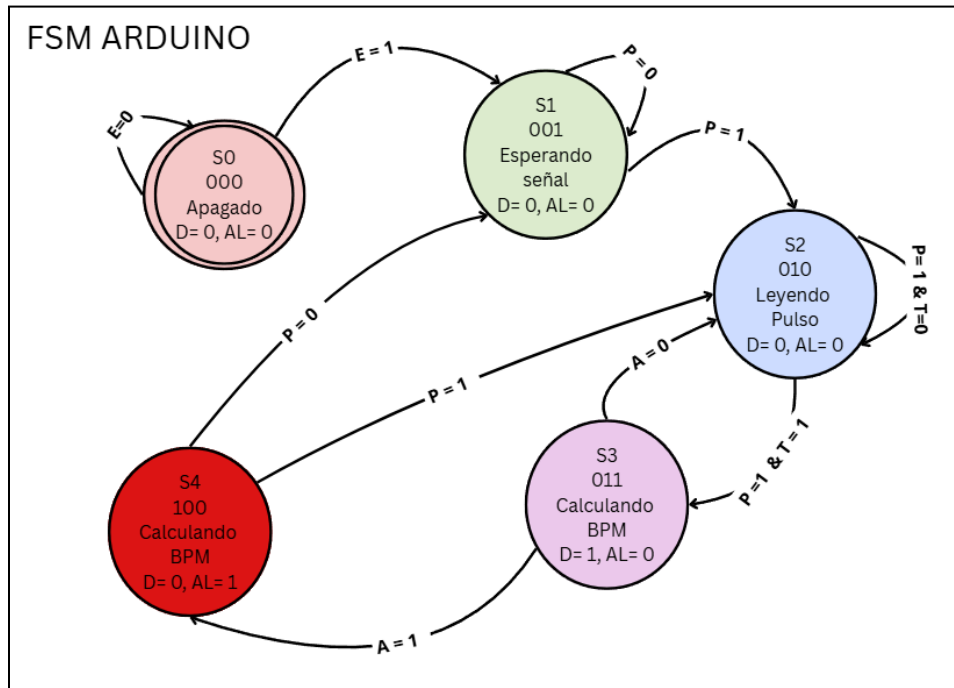
CURSO:

Sistemas Digitales y Arquitectura de Computadoras

LIMA-LIMA-PERÚ

2026

1. Diagrama de flujo o máquina de estados del firmware.



Leyenda:

1. Entradas Lógicas (Inputs)

- E = Encendido
- P = Pulso detectado
- T = Temporizador
- A = Anomalía

2. Salidas Lógicas (Outputs)

- D = Transmisión de Datos
- AL = Alerta de Emergencia

Estado Actual (S _n)	Código Binario (Q ₂ Q ₁ Q ₀)	Condiciones de Entrada Evaluadas	Estado Siguiente (S _{n+1})	Código Siguiente (Q ₂ +Q ₁ +Q ₀)	Salida D (Bluetooth)	Salida AL (Alerta)
S0 (Apagado)	000	E = 0	S0	000	0	0
	000	E = 1	S1	001	0	0
S1 (Esperando Señal)	001	P = 0	S1	001	0	0
	001	P = 1	S2	010	0	0
S2 (Leyendo Pulso)	010	P = 0	S1	001	0	0
	010	P = 1. T = 0	S2	010	0	0
	010	P = 1 . T = 1	S3	011	0	0
S3 (Calculando y Enviando)	011	A = 0	S2	010	1	0
	011	A = 1	S4	100	1	0
S4 (Alerta Crítica)	100	P = 1	S2	010	0	1

	100	P = 0	S1	001	0	1
--	-----	-------	----	-----	---	---

2. Mapa de memoria simplificado del controlador, organización de la memoria de programa y datos.

Arquitectura AVR (Basada en Harvard)

En los microcontroladores AVR, la memoria RAM estática (SRAM) se organiza de forma interna en cinco secciones principales:

- Text (Código): Contiene las instrucciones del programa que se cargan y ejecutan desde la memoria flash.
- Data (Datos): Almacena las variables globales y estáticas que ya han sido inicializadas por el usuario en el boceto (*sketch*).
- BSS: Dedicada exclusivamente a los datos y variables globales que no han sido inicializados.
- Stack (Pila): Espacio dinámico que almacena temporalmente los datos locales de las funciones, parámetros y el estado del sistema durante las interrupciones.
- Heap (Montículo): Región de memoria utilizada para almacenar variables creadas dinámicamente durante el tiempo de ejecución del programa.

Arquitectura ARM (Híbrida y Avanzada)

A diferencia de AVR, las arquitecturas ARM implementan un mapa de memoria más complejo (con configuraciones de direcciones de 32, 36 o 40 bits) que interactúa con el diseño del *System on a Chip* (SoC) y memorias DRAM adicionales.

La Unidad de Gestión de Memoria (MMU)

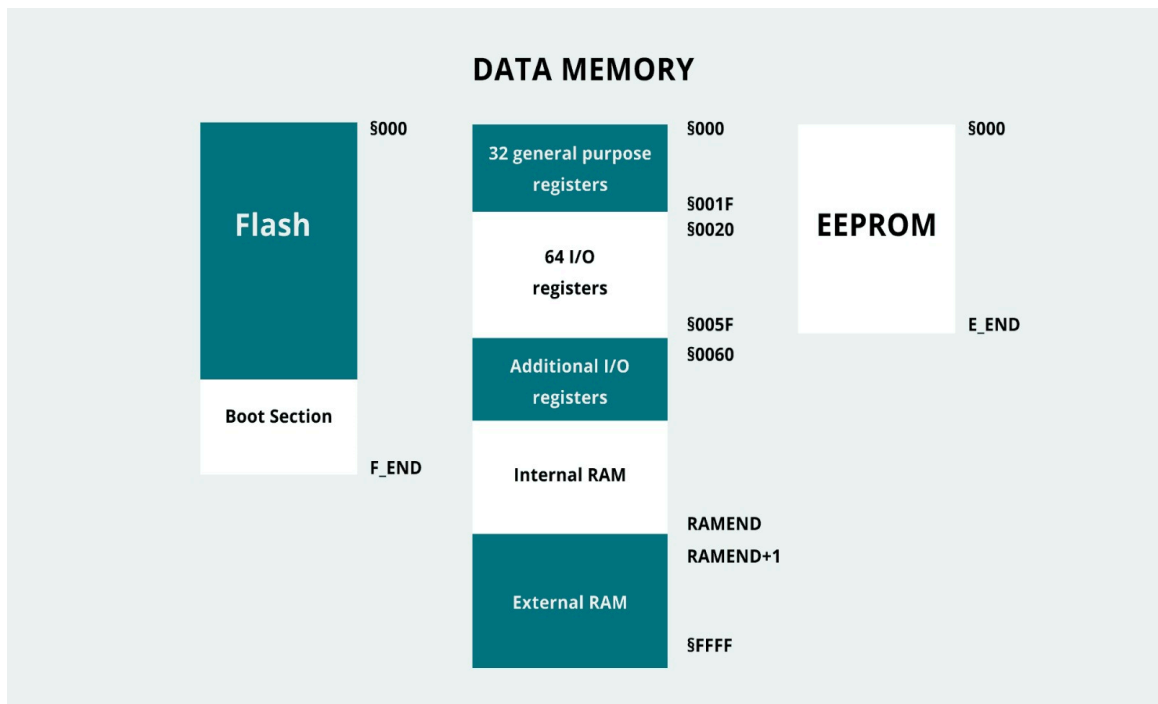
Es el núcleo del control de memoria en ARM. Su función principal es permitir la ejecución multitarea independiente, logrando que cada tarea corra en su propio espacio virtual.

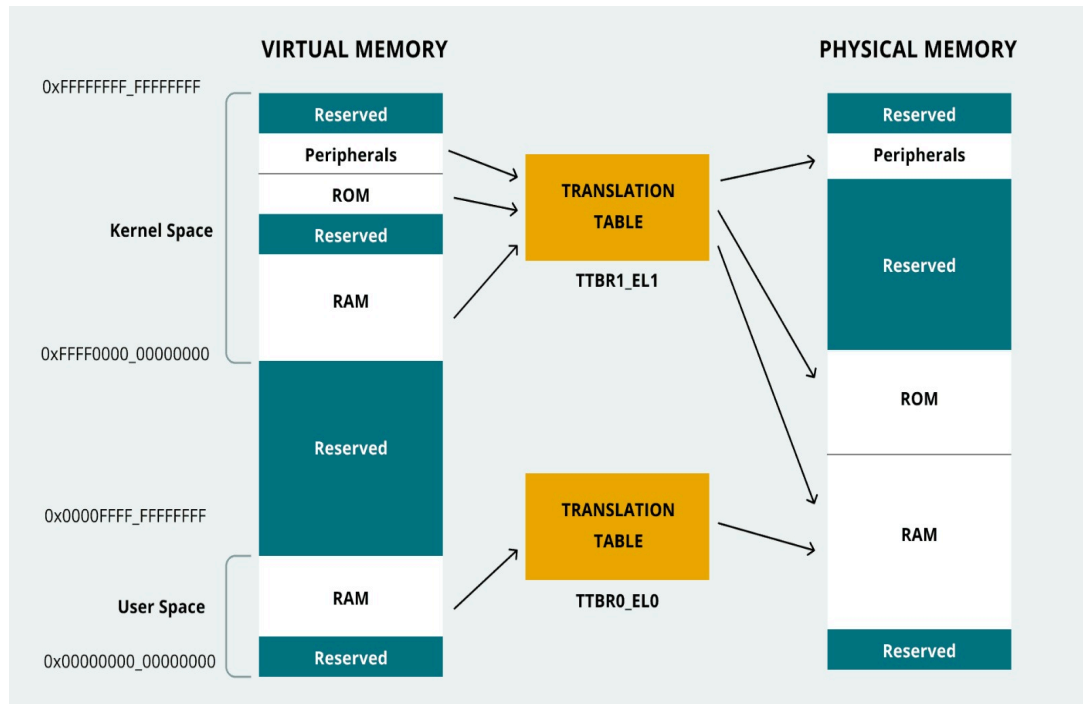
- Tablas de traducción: La MMU utiliza estas tablas para conectar e interactuar entre las direcciones virtuales (gestionadas por software a nivel de código) y las direcciones físicas (el sistema de memoria real).

Organización del Mapa de Memoria en ARM

El espacio se divide estrictamente según el tipo de dirección:

- Dirección Virtual (Nivel de Software):
 - *Kernel code and data*: Código y datos del núcleo del sistema operativo o sistema base.
 - *Application code and data*: Código y datos de las aplicaciones o bocetos del usuario.
- Dirección Física (Nivel de Hardware):
 - Componentes físicos reales: ROM, RAM, Flash y los registros de los Periféricos integrados.





3. Descripción de las interfaces de comunicación usadas y cómo se programan (registros involucrados si es bare-metal; o análisis de las librerías).

Interfaz de Comunicación	Descripción y Función en el Sistema	Librerías Utilizadas y Análisis
I2C (Inter-Integrated Circuit)	Interfaz serial síncrona que conecta periféricos usando dos líneas: SDA (Datos) y SCL (Reloj). En el sistema, se utiliza de forma nativa para comunicar el microcontrolador con el sensor de oximetría MAX30102 y la pantalla OLED SSD1306.	Es gestionada a bajo nivel por el hardware TWI del ATmega328P. El código incluye directamente los encabezados de control de periféricos (MAX30102.h y ssd1306.h). Mediante comandos I2C, se configuran los registros de corriente de los LEDs (rojo e infrarrojo) y el muestreo del sensor, además de enviar los mapas de bits y variables a la pantalla OLED mediante paginación (oled.firstPage()).

UART (Universal Asynchronous Receiver Transmitter)	Interfaz serial asíncrona punto a punto que utiliza líneas TX (Transmisión) y RX (Recepción). Se emplea en dos frentes: la comunicación hardware con la PC (Monitor Serial) y la transmisión inalámbrica de datos hacia el módulo Bluetooth.	Se utiliza la librería SoftwareSerial.h para emular un puerto serie por software en los pines 10 (RX) y 11 (TX) dedicado al módulo Bluetooth, operando a una velocidad de 9600 bps. Adicionalmente, se usa la clase nativa Serial conectada al puente USB-UART de la placa para la depuración del sistema.
Bluetooth Clásico (SPP - Serial Port Profile)	Protocolo inalámbrico de corto alcance. En el hardware no es procesado directamente por la CPU del Arduino, sino por un módulo externo (como el HC-05) que actúa como puente transparente de datos.	El módulo externo implementa el perfil SPP, convirtiendo el flujo de datos UART enviado desde el Arduino en paquetes de radiofrecuencia. El flujo de datos en el código final se transmite de forma directa mediante variables separadas de texto plano (ej. enviando cadenas de texto del pulso y de la saturación hacia la aplicación receptora).

a. Librería de Control del Sensor (MAX30102.h / Pulse.h)

A diferencia de las soluciones comerciales abstractas, este proyecto procesa los datos puros (Raw Data) del sensor **MAX30102**. El código accede mediante I2C a los registros de capturas del sensor utilizando sensor.getIR() y sensor.getRed().

- **Análisis de bajo nivel:** La librería remueve la componente continua (DC offset) de la señal fotopletimográfica (PPG) mediante código interno (pulseIR.dc_filter()) y suaviza la señal con filtros de media móvil (ma_filter) para realizar una detección matemática de picos basada en un umbral dinámico, calculando el intervalo de tiempo entre latidos mediante la función millis().

b. Librería de Visualización (ssd1306h.h / LiquidCrystal.h)

Dependiendo de la etapa del prototipo, el código integra soporte para pantallas LCD de caracteres o pantallas OLED de alta resolución basadas en el driver **SSD1306**:

- **Enfoque LCD:** Utiliza LiquidCrystal.h controlando los pines digitales mediante mapeo de bits en paralelo (RS, E, D4-D7) para enviar caracteres directos.
- **Enfoque OLED:** Utiliza ssd1306h.h interactuando mediante ráfagas de datos I2C. Para optimizar el uso de la memoria RAM del ATmega328P, emplea una técnica de

renderizado por páginas (oled.nextPage()) y almacena estructuras gráficas pesadas (como el mapa de bits del corazón heart_bits) directamente en la memoria de programa (Flash) usando el modificador PROGMEM.

c. Librería SoftwareSerial.h

Permite abrir un canal de comunicación asíncrono liberando el puerto serie principal para la depuración en la PC.

- **Análisis de Registros y Hardware:** Mientras que la UART por hardware altera directamente los registros de control del ATmega328P (UBRR0 para el Baud Rate, UCSRB para habilitar transmisión/recepción y UDR0 para el búfer de datos), SoftwareSerial emula este comportamiento manipulando pines de entrada/salida digital por software. Utiliza interrupciones por cambio de pin (PCINT) para detectar de forma precisa el bit de inicio (Start Bit) de los datos entrantes del Bluetooth, interrumpiendo brevemente el bucle principal (loop()) para realizar el conteo de tiempo crítico correspondiente a los 9600 baudios.

4. **Código fuente de la parte de adquisición y procesamiento, con comentarios que relacionan con los conceptos de arquitectura (p. ej., “aquí se configura el timer1 en modo CTC para generar interrupción cada 10 ms”).**

SECCIÓN 1: DEFINICIÓN DE BUFFER DE MEMORIA Y ARQUITECTURA HARVARD (PROGMEM)

```
// En la arquitectura Harvard del microcontrolador AVR, la memoria Flash (programa)
// y la SRAM (datos) están físicamente separadas. Usamos PROGMEM para forzar al
// compilador a almacenar esta tabla en la Flash ROM y liberar la escasa memoria
// SRAM.
```

```
const uint8_t spo2_table[184] PROGMEM = {
    95, 96, 97, 98, 99, 100 // ... [Datos de calibración del sensor]
};
```

```
// Estructura de almacenamiento en anillo (Circular Buffer) implementada en software.
// Previene el desbordamiento de memoria asignando un espacio estático fijo en la
// SRAM.
```

```
const uint8_t MAXWAVE = 72;
uint8_t waveform[MAXWAVE];
uint8_t wavep = 0; // Puntero de indexación del buffer circular
```


SECCIÓN 2: SUBSISTEMA DE ADQUISICIÓN DE DATOS (MÓDULO ANALÓGICO / BUS DE COMUNICACIÓN)

```
void adquirirDatos()
{
    // -----
    // Caso A: Si usas el Convertidor Analógico-Digital (ADC Interno del
    // Microcontrolador)
    // -----
    // Arquitectura del ADC: Aquí el multiplexor interno conecta el canal A0 al
    // Sample and Hold.
    // El registro ADCSRA inicia la aproximación sucesiva que toma 13 ciclos de
    // reloj de ADC.
    // El hilo de ejecución se bloquea en un lazo de "Polling" esperando que el
    // hardware
    // ponga a cero el bit ADSC (lectura por encuesta).
    int pulseValue = analogRead(A0);

    // -----
    // Caso B: Si usas el Sensor Digital MAX30102 (Bus de Periféricos I2C / TWI)
    // -----
    // Arquitectura del Bus: El periférico TWI (Two Wire Interface) de la CPU toma el
    // control.
    // Se genera una condición de START en las líneas de hardware SDA/SCL, se
    // transmite la
    // dirección del esclavo y se lee el buffer FIFO interno del sensor mediante
    // interrupciones del bus.
    sensor.check();
    if (!sensor.available()) return;

    // Los registros de datos del sensor (18-bits) son transferidos por el bus I2C
    // y la ALU del microcontrolador los concatena en variables de 32 bits (uint32_t)
    uint32_t irValue = sensor.getIR();
    uint32_t redValue = sensor.getRed();
    sensor.nextSample(); // Desplaza el puntero del registro FIFO del sensor por
    hardware
}
```

SECCIÓN 3: PROCESAMIENTO DIGITAL DE SEÑALES (DSP) EN LA ALU (UNIDAD ARITMÉTICA LÓGICA)

```
// Condición de Umbral de Hardware: Si la irradiancia es baja, el sensor entra en
// modo ahorro.
if (irValue < 5000) {
```

```

    // Gestión de Energía: Modifica los bits del registro SMCR (Sleep Mode
    Control Register)
    // para preparar a la CPU para ejecutar la instrucción en ensamblador
    'SLEEP'.
    // Adicionalmente, se limpia el bit ADEN en ADCSRA para apagar los
    amplificadores operacionales del ADC.
    cbi(ADCSRA, ADEN);
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sleep_mode();
}

    // Remoción de la Componente Continua (Filtro DC por Software):
    // La ALU procesa matemáticamente la señal para separar la absorción de luz
    estática (DC)
    // de la variable por el pulso arterial (AC). Requiere operaciones de 16 bits de
    precisión.
    int16_t IR_signal = pulseIR.dc_filter(irValue);
    int16_t Red_signal = pulseRed.dc_filter(redValue);

    // Filtro de Media Móvil (Moving Average Filter):
    // Implementación de un filtro FIR (Finite Impulse Response) en software. La
    CPU almacena un
    // arreglo de muestras previas en la SRAM y la ALU calcula el promedio
    ponderado.
    // Esto elimina el ruido de alta frecuencia del entorno (ruido de la red eléctrica
    de 50Hz/60Hz).
    int16_t IR_filtrado = pulseIR.ma_filter(IR_signal);
    int16_t Red_filtrado = pulseRed.ma_filter(Red_signal);

```

SECCIÓN 4: EXTRACCIÓN DE CARACTERÍSTICAS Y MANEJO DEL TIEMPO (TIMERS DE LA CPU)

```

    // Algoritmo de Detección de Picos (Peak Detection):
    // Compara la muestra actual con el umbral dinámico calculado. Si es mayor, se
    determina un latido.
    if (pulseIR.isBeat(IR_filtrado))
    {
        // Temporización por Hardware (Uso del Timer0 Interno):
        // La función 'millis()' lee una variable en la SRAM que es incrementada
        automáticamente
        // cada 1.024 milisegundos por la Rutina de Servicio de Interrupción (ISR) del
        Timer0.
        // Aquí la ALU calcula el intervalo R-R (tiempo entre latidos) mediante una
        resta de registros.
    }

```

```

long now = millis();
long beatInterval = now - lastBeatTime;
lastBeatTime = now;

// Prevención de rebotes por Software (Debounce temporal):
// Si el intervalo es menor a 300 ms (equivalente a un límite
físico-arquitectónico de >200 BPM),
// la CPU descarta la muestra para evitar falsos positivos generados por ruido
electromagnético.
if (beatInterval > 300)
{
    // Operación de división en la ALU: Como el microcontrolador AVR de 8 bits
    no tiene una
    // instrucción de división por hardware en su set de instrucciones RISC, el
    compilador
    // genera un subprograma iterativo pesado en la memoria para calcular los
    BPM.
    int beatsPerMinute = 60000 / beatInterval;
}

```

SECCIÓN 5: CÁLCULO BIOMÉDICO Y ACCESO AL BUS DE PROGRAMA (LPM)

```

// Algoritmo Ratio-of-Ratios para SpO2:
// Se calculan los valores eficaces (AC y DC) de ambos canales ópticos.
long numerator = (pulseRed.avgAC() * pulseIR.avgDC()) / 256;
long denominator = (pulseRed.avgDC() * pulseIR.avgAC()) / 256;
int RX100 = (denominator > 0) ? (numerator * 100) / denominator : 999;

// Acceso indexado a la memoria de Programa:
// Para calcular el porcentaje final de SpO2 sin usar operaciones de punto
flotante (floats)
// que destruirían el tiempo de ciclo de la CPU, se realiza un
direccionamiento indirecto.
// La instrucción de bajo nivel 'LPM' (Load Program Memory) toma el valor de
'RX100' como
// un desplazamiento (offset) en el bus de direcciones Z de la Flash ROM.
if ((RX100 >= 0) && (RX100 < 184)) {
    int SPO2 = pgm_read_byte_near(&spo2_table[RX100]);
}
}
}
}
}

```

5. Evidencia de Funcionamiento:

